HITEC UNIVERSITY TAXILA

OPERATING SYSTEM

SEMESTER 05
BSCS "C"

PROJECT REPORT

SUBMITTED TO:

SIR HUSSNAIN

SUBMITTED BY:

EHRAM (22-CS-O27)

TOPIC:

Producer Consumer Example Implementation

Implement Producer Consumer Example in Operating System.

Report on Producer-Consumer Problem Code:

---

1. Introduction

The Producer-Consumer problem is a classic synchronization problem that illustrates the challenges of managing shared resources in a multi-threaded environment. It involves two types of processes:

- **Producer**: Adds items to a shared buffer.
- **Consumer**: Removes items from the buffer.

The problem demonstrates issues like race conditions when multiple processes access the buffer without proper synchronization mechanisms. This report discusses the code implementation provided, which includes an unsynchronized demonstration of the problem and a synchronized solution using basic concepts.

---

2. Objective

The main objective of this code is to:

1. Demonstrate the problems arising from the lack of synchronization in the Producer-Consumer scenario.
2. Provide a synchronized solution to prevent issues like buffer overflow and underflow.

---

3. Methodology

The implementation consists of two main functions:

1. **Demonstrating the Problem**: Shows the issues without synchronization, highlighting the need for proper mechanisms.
2. **Providing the Solution**: Implements synchronization using simple variables to manage buffer states and ensure thread safety.

The program uses a menu-driven approach to allow users to choose between the demonstration and the solution.

---

4. Explanation of Code

Variables:

- **buffer**: Tracks the number of items currently in the buffer.
- **BUFFER_SIZE**: Defines the maximum capacity of the buffer.
- **mutex**: Simulates a mutual exclusion lock to prevent race conditions.
- **full**: Represents the number of occupied slots in the buffer.
- **empty**: Represents the number of available slots in the buffer.

Main Functions:

4.1 Problem Demonstration

This function highlights the absence of synchronization. Users can:

- Produce items until the buffer overflows.
- Consume items until the buffer underflows.

4.2 Solution with Synchronization

This function uses basic synchronization:

- **Producer** waits if the buffer is full.
- **Consumer** waits if the buffer is empty.
- **mutex** ensures only one process accesses the buffer at a time.

Menu:

- The user can choose between demonstrating the problem, viewing the solution, or exiting the program.

---

5. Code

TEXT FORM OF CODE:

```
#include <iostream>

#include <cstdlib> // For exit()
```

```
using namespace std;

// Variables for buffer status
int buffer = 0;
const int BUFFER_SIZE = 10;

// Function to demonstrate the producer-consumer problem (unsynchronized)
void problem()
{
    cout << "\n=== Producer-Consumer Problem Demonstration ===";
    cout << "\nThis demonstrates the issue without synchronization.";
    cout << "\n1. Producer tries to add items to the buffer.";
    cout << "\n2. Consumer tries to remove items from the buffer.";
    cout << "\n3. Without proper synchronization, race conditions can occur.";
    cout << "\n\nLet's simulate this:\n";

    char alphabet;
    for (int i = 0; i < 10; i++) {
        cout << "Enter 'p' to produce or 'c' to consume (any other key to stop): ";
        cin >> alphabet;

        if (alphabet == 'p') {
            if (buffer < BUFFER_SIZE) {
                buffer++;
                cout << "Produced an item. Items in buffer: " << buffer << endl;
            } else {
```

```
            cout << "Buffer overflow! Cannot produce more items.\n";
        }
    } else if (alphabet == 'c') {
        if (buffer > 0) {
            buffer--;
            cout << "Consumed an item. Items in buffer: " << buffer << endl;
        } else {
            cout << "Buffer underflow! Nothing to consume.\n";
        }
    } else {
        cout << "Stopping simulation of the problem.\n";
        break;
    }
  }
}

// Function to solve the producer-consumer problem using synchronization
void solution()
{
    cout << "\n=== Producer-Consumer Problem Solution ===";
    cout << "\nThis demonstrates synchronization to solve the issue.";
    cout << "\n1. Producer waits if the buffer is full.";
    cout << "\n2. Consumer waits if the buffer is empty.";
    cout << "\n3. Access to the buffer is synchronized.\n";

    int mutex = 1, full = 0, empty = BUFFER_SIZE;
```

```
char action;

while (true) {
    cout << "\nEnter 'p' to produce, 'c' to consume, or 'q' to quit: ";
    cin >> action;

    if (action == 'p') {
        if (mutex == 1 && empty > 0) {
            mutex--;
            full++;
            empty--;
            cout << "Produced an item. Items in buffer: " << full << endl;
            mutex++;
        } else {
            cout << "Buffer is full! Producer is waiting...\n";
        }
    } else if (action == 'c') {
        if (mutex == 1 && full > 0) {
            mutex--;
            full--;
            empty++;
            cout << "Consumed an item. Items in buffer: " << full << endl;
            mutex++;
        } else {
            cout << "Buffer is empty! Consumer is waiting...\n";
        }
```

```
        } else if (action == 'q') {
            cout << "Exiting the solution simulation...\n";
            break;
        } else {
            cout << "Invalid choice! Please enter 'p', 'c', or 'q'.\n";
        }
    }
}

// Driver Code
int main()
{
    int choice;
    cout << "\n=== Producer-Consumer Problem Menu ===";
    cout << "\n1. Demonstrate the Problem";
    cout << "\n2. Provide the Solution";
    cout << "\n3. Exit";

    while (true) {
        cout << "\nEnter your choice: ";
        cin >> choice;

        switch (choice) {
        case 1:
            problem();
            break;
```

```
        case 2:
            solution();
            break;

        case 3:
            cout << "Exiting program. Goodbye!\n";
            exit(0);

        default:
            cout << "Invalid choice! Please try again.\n";
        }
    }
    return 0;
}
```

SCREENSHOT OF CODE:

C:\Users\hp\Documents\producer consumer final.cpp - [Executing] - Embarcadero Dev-C++ 6.3

File Edit Search View Project Execute Tools AStyle Window Help

TDM-GCC 9.2.0 64-bit Release

(globals)

Project Clas producer consumer final.cpp

```cpp
#include <iostream>
#include <cstdlib>
using namespace std;

int buffer = 0;
const int BUFFER_SIZE = 10;

void problem()
{
    cout << "\n=== Producer-Consumer Problem Demonstration ===";
    cout << "\nThis demonstrates the issue without synchronization.";
    cout << "\n1. Producer tries to add items to the buffer.";
    cout << "\n2. Consumer tries to remove items from the buffer.";
    cout << "\n3. Without proper synchronization, race conditions can occur.";
    cout << "\n\nLet's simulate this:\n";

    char alphabet;
    for (int i = 0; i < 10; i++) {
        cout << "Enter 'p' to produce or 'c' to consume (any other key to stop): ";
        cin >> alphabet;

        if (alphabet == 'p') {
            if (buffer < BUFFER_SIZE) {
                buffer++;
                cout << "Produced an item. Items in buffer: " << buffer << endl;
            } else {
                cout << "Buffer overflow! Cannot produce more items.\n";
            }
        } else if (alphabet == 'c') {
            if (buffer > 0) {
                buffer--;
                cout << "Consumed an item. Items in buffer: " << buffer << endl;
```

Compiler (3) Resources Compile Log Debug Find Results Console Close

Abort Compilation

Shorten compiler pat

```
- Errors: 0
- Warnings: 0
- Output Filename: C:\Users\hp\Documents\producer consumer final.exe
- Output Size: 2.99082469940186 MiB
- Compilation Time: 0.58s
```

producer consumer final.cpp

```cpp
                cout << "Consumed an item. Items in buffer: " << buffer << endl;
            } else {
                cout << "Buffer underflow! Nothing to consume.\n";
            }
        } else {
            cout << "Stopping simulation of the problem.\n";
            break;
        }
    }
}

void solution()
{
    cout << "\n=== Producer-Consumer Problem Solution ===";
    cout << "\nThis demonstrates synchronization to solve the issue.";
    cout << "\n1. Producer waits if the buffer is full.";
    cout << "\n2. Consumer waits if the buffer is empty.";
    cout << "\n3. Access to the buffer is synchronized.\n";

    int mutex = 1, full = 0;
    int empty = BUFFER_SIZE;
    char action;

    while (true) {
        cout << "\nEnter 'p' to produce, 'c' to consume, or 'q' to quit: ";
        cin >> action;

        if (action == 'p') {
            if (mutex == 1 && empty > 0)
            {
                mutex--;
                full++;
```

[*] producer consumer final.cpp

```cpp
                full++;
                empty--;
                cout << "Produced an item. Items in buffer: " << full << endl;
                mutex++;
            } else {
                cout << "Buffer is full! Producer is waiting...\n";
            }
        } else if (action == 'c') {
            if (mutex == 1 && full > 0)
            {
                mutex--;
                full--;
                empty++;
                cout << "Consumed an item. Items in buffer: " << full << endl;
                mutex++;
            } else {
                cout << "Buffer is empty! Consumer is waiting...\n";
            }
        } else if (action == 'q') {
            cout << "Exiting the solution simulation...\n";
            break;
        } else {
            cout << "Invalid choice! Please enter 'p', 'c', or 'q'.\n";
        }
    }
}


int main()
{
    int choice;
    cout << "\n=== Producer-Consumer Problem Menu ===";
```

[*] producer consumer final.cpp

```cpp
int main()
{
    int choice;
    cout << "\n=== Producer-Consumer Problem Menu ===";
    cout << "\n1. Demonstrate the Problem";
    cout << "\n2. Provide the Solution";
    cout << "\n3. Exit";

    while (true) {
        cout << "\nEnter your choice: ";
        cin >> choice;

        switch (choice) {
        case 1:
            problem();
            break;

        case 2:
            solution();
            break;

        case 3:
            cout << "Exiting program. Goodbye!\n";
            exit(0);

        default:
            cout << "Invalid choice! Please try again.\n";
        }
    }
    return 0;
}
```

OUTPUT:

```
C:\Users\hp\Documents\prod

=== Producer-Consumer Problem Menu ===
1. Demonstrate the Problem
2. Provide the Solution
3. Exit
Enter your choice: 1

=== Producer-Consumer Problem Demonstration ===
This demonstrates the issue without synchronization.
1. Producer tries to add items to the buffer.
2. Consumer tries to remove items from the buffer.
3. Without proper synchronization, race conditions can occur.

Let's simulate this:
Enter 'p' to produce or 'c' to consume (any other key to stop): p
Produced an item. Items in buffer: 1
Enter 'p' to produce or 'c' to consume (any other key to stop): p
Produced an item. Items in buffer: 2
Enter 'p' to produce or 'c' to consume (any other key to stop): p
Produced an item. Items in buffer: 3
Enter 'p' to produce or 'c' to consume (any other key to stop): p
Produced an item. Items in buffer: 4
Enter 'p' to produce or 'c' to consume (any other key to stop): c
Consumed an item. Items in buffer: 3
Enter 'p' to produce or 'c' to consume (any other key to stop): c
Consumed an item. Items in buffer: 2
Enter 'p' to produce or 'c' to consume (any other key to stop): 1
Stopping simulation of the problem.

Enter your choice: 2
```

```
C:\Users\hp\Documents\prod

Enter your choice: 2

=== Producer-Consumer Problem Solution ===
This demonstrates synchronization to solve the issue.
1. Producer waits if the buffer is full.
2. Consumer waits if the buffer is empty.
3. Access to the buffer is synchronized.

Enter 'p' to produce, 'c' to consume, or 'q' to quit: pp
Produced an item. Items in buffer: 1

Enter 'p' to produce, 'c' to consume, or 'q' to quit: Produced an item. Items in buffer: 2

Enter 'p' to produce, 'c' to consume, or 'q' to quit: p
Produced an item. Items in buffer: 3

Enter 'p' to produce, 'c' to consume, or 'q' to quit: p
Produced an item. Items in buffer: 4

Enter 'p' to produce, 'c' to consume, or 'q' to quit: p
Produced an item. Items in buffer: 5

Enter 'p' to produce, 'c' to consume, or 'q' to quit: p
Produced an item. Items in buffer: 6

Enter 'p' to produce, 'c' to consume, or 'q' to quit: p
Produced an item. Items in buffer: 7

Enter 'p' to produce, 'c' to consume, or 'q' to quit: p
Produced an item. Items in buffer: 8
```

```
C:\Users\hp\Documents\prod

Enter 'p' to produce, 'c' to consume, or 'q' to quit: p
Produced an item. Items in buffer: 8

Enter 'p' to produce, 'c' to consume, or 'q' to quit: p
Produced an item. Items in buffer: 9

Enter 'p' to produce, 'c' to consume, or 'q' to quit: p
Produced an item. Items in buffer: 10

Enter 'p' to produce, 'c' to consume, or 'q' to quit: p
Buffer is full! Producer is waiting...

Enter 'p' to produce, 'c' to consume, or 'q' to quit: p
Buffer is full! Producer is waiting...

Enter 'p' to produce, 'c' to consume, or 'q' to quit: c
Consumed an item. Items in buffer: 9

Enter 'p' to produce, 'c' to consume, or 'q' to quit: c
Consumed an item. Items in buffer: 8

Enter 'p' to produce, 'c' to consume, or 'q' to quit: c
Consumed an item. Items in buffer: 7

Enter 'p' to produce, 'c' to consume, or 'q' to quit: c
Consumed an item. Items in buffer: 6

Enter 'p' to produce, 'c' to consume, or 'q' to quit: c
Consumed an item. Items in buffer: 5
```

```
C:\Users\hp\Documents\prod

Enter 'p' to produce, 'c' to consume, or 'q' to quit: c
Consumed an item. Items in buffer: 5

Enter 'p' to produce, 'c' to consume, or 'q' to quit: c
Consumed an item. Items in buffer: 4

Enter 'p' to produce, 'c' to consume, or 'q' to quit: c
Consumed an item. Items in buffer: 3

Enter 'p' to produce, 'c' to consume, or 'q' to quit: c
Consumed an item. Items in buffer: 2

Enter 'p' to produce, 'c' to consume, or 'q' to quit: c
Consumed an item. Items in buffer: 1

Enter 'p' to produce, 'c' to consume, or 'q' to quit: c
Consumed an item. Items in buffer: 0

Enter 'p' to produce, 'c' to consume, or 'q' to quit: c
Buffer is empty! Consumer is waiting...

Enter 'p' to produce, 'c' to consume, or 'q' to quit: q
Exiting the solution simulation...

Enter your choice: 3
Exiting program. Goodbye!

--------------------------------
Process exited after 100.8 seconds with return value 0
```

6. Conclusion

This implementation effectively demonstrates the challenges and solutions of the Producer-Consumer problem. The problem function highlights the importance of synchronization, while the solution implements basic mechanisms to address these issues. This code provides a foundation for understanding synchronization concepts, paving the way for more complex implementations using advanced synchronization tools like semaphores or mutex locks in multithreaded environments.

==========